

ISM3232 — Week 3 Lab

Virtual Environments & Shell Customisation — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 03 · Unit 1 · Developer Foundations

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Timing Plan (75 minutes)	3
6 Segment-by-Segment Walkthrough	5
6.1 Intro (0:00–0:04)	5
6.2 Part 1 — Virtual Environment Workflow (0:04–0:22, 18 min)	5
6.3 Part 2 — .gitignore (0:22–0:32, 10 min)	8
6.4 Part 3 — .zshrc Aliases and Functions (0:32–0:52, 20 min)	10
6.5 Part 4 — Practice and Deactivate (0:52–1:02, 10 min)	13
6.6 Stretch — Recreate the Environment from requirements.txt (1:02–1:15, as time allows)	15
7 Wrap-Up (last ~13 minutes, folded into Stretch time if unused)	16
8 Appendix A — Full Command Reference	16
9 Appendix B — Extra Practice (only if the class finishes early)	17

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 3 Lab — Virtual Environments & Shell Customisation
Unit	Unit 1 · Developer Foundations
Class length	Full class period (75 minutes)
Format	Live terminal code-along
Prerequisites	Weeks 1–2: verified environment, <code>ism3232/</code> structure, core navigation/file commands (marked Midterm-Eligible)
Student-facing lab page	Week 3 In-Class Lab — Module 2C & 2D, “Virtual Environments and <code>.zshrc</code> ”
Parts covered	Part 1 (venv workflow) – Part 4 (practice + deactivate) + Stretch (rebuild from <code>requirements.txt</code>)
Submission	4 screenshots + <code>requirements.txt</code> + <code>.gitignore</code> + updated <code>README.md</code> , Canvas, completion credit

Two genuinely separate ideas share this lab, and it’s worth naming that split to the class up front: **virtual environments** (an isolated package “room” per project) and **shell customization** (`.zshrc` aliases and functions that make the terminal itself faster to use). Neither depends on the other, but both are foundational habits the rest of the semester assumes are already automatic — Week 4’s Git ritual uses the aliases defined today (`gs`, `ga`, `gcmmsg`, `gp`) as shorthand, and every remaining Python module runs inside a project-specific `.venv`. The lab page’s own mental model — “a venv is a private room for each project” — is worth repeating verbatim; it’s the single clearest explanation of *why* this matters, not just *how*.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Create and activate a Python virtual environment, and explain what “isolated” actually means in concrete, checkable terms (a different `which python3`, a different `pip list`).
2. Install packages inside an active venv, and freeze the exact installed versions to `requirements.txt` for reproducibility.
3. Explain why `.venv/` must never be committed to Git, and create a `.gitignore` that excludes it before any files are staged.

4. Add custom aliases and a shell function to `.zshrc`, apply the changes without restarting the terminal, and verify each one works.
5. Deactivate a virtual environment cleanly and confirm the shell has returned to the system Python.

3 Before Class — Setup Checklist

- ☐ Rehearse the full `venv` → `install` → `freeze` → `gitignore` → `alias` → `deactivate` sequence yourself before class, end to end, on your demo machine — this lab has more sequential dependencies than Weeks 1–2 (a broken `venv` early on cascades into every later part failing), so a dry run matters more here.
- ☐ Decide how you'll demonstrate Part 1's "isolation" verification — opening a genuine second terminal tab, side by side with the `venv`-activated one, is the clearest way to make "isolated" concrete rather than asserted; confirm your screen-share setup can show both at once if possible.
- ☐ Back up (or know how to restore) your own `~/ .zshrc` before editing it live in front of the room — a good moment to model the exact caution you're about to ask students to exercise ("do not delete anything that is already there").

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, from Weeks 1–2's verified setup
- Students: their existing `ism3232/module02_zsh/` folder
- Internet access for `pip install`

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: the "private room" mental model	4
0:04–0:22	Part 1 — Virtual environment workflow	18
0:22–0:32	Part 2 — <code>.gitignore</code>	10
0:32–0:52	Part 3 — <code>.zshrc</code> aliases and functions	20
0:52–1:02	Part 4 — Practice and deactivate + README update	10

Time	Segment	Minutes
1:02–1:15	Stretch (rebuild from <code>requirements.txt</code>) + wrap-up, submission checklist	13

Four required parts fill the bulk of the class period; the Stretch (destroying and rebuilding the `venv` from `requirements.txt`) is given real time if the room reaches it, since it's the most concrete possible demonstration that `requirements.txt` actually captures everything needed to reproduce the environment — worth doing live, not just described, if time allows.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“Today has two halves that don’t depend on each other, but both become automatic habits for the rest of the semester. First half: virtual environments — think of a venv as a private room for one project. When you activate it, you walk in, and every package you install goes on that room’s shelves only. Walk out — deactivate — and other projects are completely unaffected. Second half: customizing your shell itself, so common commands become one or two characters instead of a full phrase.”

Do: Write the “private room” analogy on the board and leave it visible through Part 1 — it’s worth returning to explicitly once isolation is demonstrated concretely.

6.2 Part 1 — Virtual Environment Workflow (0:04–0:22, 18 min)

Teaching goal: Create, activate, and populate a venv — and, critically, *prove* its isolation concretely, not just assert it.

Say to the class:

“Three steps: create, activate, verify it’s actually isolated — not just trust that it is.”

Live-code this:

```
cd ~/ism3232/module02_zsh
python3 -m venv .venv
source .venv/bin/activate
# Prompt should now show (.venv)

which python3    # should point inside .venv/
pip list         # should show minimal packages
```

Line-by-line explanation:

- `python3 -m venv .venv` — `-m venv` runs Python’s built-in `venv` module, which creates a new, self-contained Python environment inside a folder named `.venv` (the leading dot makes it hidden, per Week 2’s `ls -la` lesson). Say explicitly: this folder now contains its **own copy** of the Python interpreter and its own separate place to install packages — nothing installed here touches the system-wide Python.
- `source .venv/bin/activate` — **this is the step that actually “walks into the room.”** `source` runs a script in the *current* shell session (rather than a separate subprocess, which is

why the effect persists in your terminal afterward) — the activation script modifies your shell's PATH so that python3 and pip now resolve to the venv's copies first, ahead of the system versions.

- The (.venv) prefix appearing in the prompt — a visual confirmation, built into most shell configurations, that a venv is currently active; say explicitly this is worth training yourself to glance at reflexively, the same way Week 1 trained checking for % vs \$.
- which python3 — Week 1's command, now revealing something new: instead of a system path like /usr/bin/python3 or /opt/homebrew/bin/python3, it should now point *inside* the .venv folder itself (e.g., .../module02_zsh/.venv/bin/python3) — this is the first concrete, checkable proof that activation actually changed something.
- pip list — should show only a minimal handful of packages (pip itself, maybe setuptools) — a fresh venv starts essentially empty, deliberately, so whatever gets installed next is fully visible and attributable to this specific project.

Verified example output:

```
/private/tmp/.../module02_zsh/.venv/bin/python3
Package Version
-----
pip      24.0
```

Now install and freeze:

```
pip install pytest ruff
pip list                      # confirm both are installed
pip freeze > requirements.txt
cat requirements.txt          # confirm it was written
```

Line-by-line explanation:

- pip install pytest ruff — installs two packages this course uses constantly from here forward: pytest (testing, first used next module) and ruff (formatting/linting, first used next module too) — both land inside .venv/, not system-wide.
- pip freeze > requirements.txt — **pip freeze** lists every installed package *with its exact version number* (e.g., pytest==9.1.1, not just pytest); > (Week 2's redirect operator) writes that list to a file instead of printing it. Say explicitly why exact versions matter, not just names: **requirements.txt is what lets anyone — a classmate, a grader, or you on a different machine — recreate this exact environment**, down to the specific version of every package, which the Stretch section demonstrates directly.

Verified requirements.txt content:

```
iniconfig==2.3.0
packaging==26.3
pluggy==1.6.0
Pygments==2.21.0
```

```
pytest==9.1.1
ruff==0.16.4
```

Point out explicitly: more than just `pytest` and `ruff` appear — those two packages themselves depend on smaller supporting packages (`pluggy`, `iniconfig`, `Pygments`), which `pip` installed automatically. `pip freeze` captures the *entire* resulting environment, dependencies included, not just what was explicitly typed.

Now verify isolation, concretely — open a second terminal tab, without activating the venv:

```
which python3      # points to system Python
pip list           # does NOT include pytest and ruff
```

Say explicitly, since this is the exercise's real payoff: “This second terminal never ran `source .venv/bin/activate` — and `which python3` proves it's using a completely different interpreter, and `pip list` proves `pytest/ruff` simply aren't there. This is what ‘isolated’ concretely means: not a promise, a *checkable fact* about two different terminal sessions on the same machine, right now.”

Common student mistakes to watch for:

- Running `pip install` **before** activating the venv — installs to the *system* Python instead, defeating the entire point; if a student's `which python3` in Part 1 didn't show a `.venv` path, stop and fix activation before proceeding to install anything.
- Forgetting `source` and just running `.venv/bin/activate` directly — this often runs the script in a subprocess that exits immediately, leaving the current shell completely unaffected; the `(.venv)` prompt prefix simply won't appear, which is the visual tell something's wrong.
- Checking isolation in the **same** terminal tab instead of a genuinely separate one — this doesn't actually test anything, since the same terminal still has the venv active; the isolation check specifically requires a fresh, non-activated session.

Check for understanding: “If you closed this terminal entirely and opened a brand new one tomorrow, would the venv still be active?” (No — activation is a per-session shell state, not a permanent setting; `source .venv/bin/activate` needs to be re-run every time you open a new terminal and want to work on this project. This is worth stating explicitly, since it's a genuine, recurring point of confusion — “why isn't my venv active anymore” is one of the most common beginner Python questions.)

6.3 Part 2 — .gitignore (0:22–0:32, 10 min)

Teaching goal: Create a .gitignore **before** any files are committed, specifically to keep .venv/ out of version control — and understand *why* committing a venv is a genuine mistake, not just a style preference.

Say to the class:

“One rule, stated as absolutely as anything in this course: .venv/ must never be committed to Git. We’re creating the .gitignore now, before Week 4’s first commit, specifically so this mistake never happens.”

Live-code this:

```
cd ~/ism3232/module02_zsh
echo '.venv/' > .gitignore
echo '__pycache__/' >> .gitignore
echo '*.pyc' >> .gitignore
cat .gitignore          # confirm all three lines
ls -la                  # .gitignore and .venv should both appear
```

Line-by-line explanation:

- `echo '.venv/' > .gitignore` — Week 2’s redirect operator, creating a brand-new .gitignore file with one line — note the **single** `>` here, correctly, since this is the *first* line and there’s nothing to preserve yet.
- `echo '__pycache__/' >> .gitignore` and `echo '*.pyc' >> .gitignore` — Week 2 Stretch’s **append** operator, `>>`, used correctly here specifically *because* the file already has content from the line above that must be preserved, not overwritten. Ask the room explicitly: “what would happen if I’d used `>` instead of `>>` on these last two lines?” (Each would have erased the previous line, leaving .gitignore with only the single most recent entry — a good, concrete callback to last week’s overwrite-vs-append distinction, now with real consequences.)
- **Why .venv/ specifically must never be committed** — worth stating plainly, since this is a real, substantive reason, not an arbitrary rule: a venv folder can be genuinely large (hundreds of megabytes with enough packages), is entirely regeneratable from `requirements.txt` in seconds, and — critically — often contains **machine-specific paths and binaries** that won’t even work correctly on a different computer or operating system. Committing it bloats the repository with something that provides zero value to a collaborator and can actively break on their machine.
- `__pycache__/` and `*.pyc` — Python’s automatically-generated compiled bytecode files (the same category flagged in comparable courses’ Module 03 .gitignore exercises) — pure clutter, regenerated automatically, never something a human wrote.

Verified .gitignore content:


```
.venv/  
__pycache__/  
*.pyc
```

Common student mistakes to watch for:

- Creating `.gitignore` **after** already running `git add .` on a prior, unrelated commit that included `.venv/` — once a file is tracked by Git, adding it to `.gitignore` afterward does *not* automatically untrack it; this requires an additional `git rm -r --cached .venv` step, not covered in this lab. This is exactly why the lab’s sequencing (`.gitignore` in Week 3, before Week 4’s first commit) matters — say explicitly that order is deliberate, not incidental.
- Typing `.venv` without the trailing slash — works in most cases since Git’s pattern matching is reasonably permissive, but the trailing `/` explicitly says “this pattern only matches a directory,” which is worth using as the more precise, professional habit.

Check for understanding: “If a classmate cloned your GitHub repo and it had properly excluded `.venv/`, what would they need to run to get a working environment on their own machine?” (`python3 -m venv .venv`, `source .venv/bin/activate`, then `pip install -r requirements.txt` — exactly the Stretch section’s sequence; getting a student to connect today’s `.gitignore` decision to that reconstruction process confirms the “why” actually landed, not just the “how.”)

6.4 Part 3 — .zshrc Aliases and Functions (0:32–0:52, 20 min)

Teaching goal: Add custom aliases and a shell function to `.zshrc`, apply changes without restarting the terminal, and verify each one — building the exact shorthand vocabulary Week 4’s Git ritual will lean on.

Say to the class:

“Second half of today: customizing the shell itself. `.zshrc` is a configuration file that runs every time you open a new terminal — anything you put here becomes available every session, automatically. We’re adding nine shortcuts today, and I want every single one tested before we move on, not just typed and assumed correct.”

Live-code this:

code `~/ .zshrc`

Add these lines — do not delete anything already there:

```
# ISM3232 required aliases
alias ll='ls -la'
alias c='clear'
alias py='python3'
alias gs='git status'
alias ga='git add .'
alias gcmsg='git commit -m'
alias gp='git push'
alias gl='git log --oneline'
alias tree2='tree -L 2'

# Required shell function
mkcd() { mkdir -p "$1" && cd "$1" }
```

Line-by-line explanation:

- `alias ll='ls -la'` — an **alias** is a simple text substitution: typing `ll` from now on runs `ls -la` exactly as if you’d typed it out. Say explicitly: this is purely a shorthand, not new functionality — every alias here just saves keystrokes on a command you already know.
- `alias gs='git status', ga='git add .', gcmsg='git commit -m', gp='git push', gl='git log --oneline'` — **these five specifically are the Git shorthand Week 4’s ritual will use** — flag this connection explicitly now, since it won’t be obvious why these particular five were chosen until next week’s lab makes them load-bearing.
- `mkcd() { mkdir -p "$1" && cd "$1" }` — **this is a shell function, not an alias**, and worth explaining as a genuinely different mechanism: a function can take an **argument** (`$1`, meaning “the first word typed after `mkcd`”) and run multiple commands using it. Walk through

it explicitly: `mkdir -p "$1"` creates a folder named whatever was typed after `mkcd` (the `-p` flag, new here, means “create any necessary parent folders too, and don’t error if it already exists”); `&&` means “run the next command only if the first one succeeded”; `cd "$1"` then moves into that same newly created folder. Say plainly: **this single command replaces the two-step `mkdir foldername` then `cd foldername` pattern used in every single lab so far this semester** — a genuine, meaningful convenience, not just a shorter alias.

Save the file, then apply the changes:

```
source ~/.zshrc
```

```
# Test each alias
```

```
ll                # should work as ls -la
gs                # should run git status
mkcd testdir      # should create and enter testdir/
pwd               # confirm you are inside testdir/
```

Line-by-line explanation of applying and testing:

- `source ~/.zshrc` — **this is the step students most often forget, and it’s worth stating explicitly why it’s required.** `.zshrc` only runs *automatically* when a **new** terminal session starts — since this terminal was already open when the file was edited, the new aliases don’t exist in it yet until `source` explicitly re-runs the file’s contents into the current session. Say plainly: closing and reopening the terminal would achieve the same effect, but `source` is faster and doesn’t lose your current location/history.
- `ll` — tests the simplest alias first; if this doesn’t work, `source ~/.zshrc` likely didn’t run successfully, or there’s a typo in the `.zshrc` edit itself — worth checking before testing the rest.
- `mkcd testdir` then `pwd` — the real test of the function: `pwd` should now show a path ending in `.../testdir`, proving both that the folder was created *and* that `cd` genuinely happened inside the same function call.

Verified output: `pwd` after `mkcd testdir` ends in `/testdir`, confirming the function performed both steps correctly.

Common student mistakes to watch for:

- Deleting or overwriting existing `.zshrc` content while adding the new lines — say explicitly, again, since this is a real risk when editing a config file for the first time: **append these lines, don’t replace the file’s existing content.** If a student is unsure what was already there, having them run `cat ~/.zshrc` before editing, to see the starting state, is a good safety step.
- Forgetting `source ~/.zshrc` after saving, then being confused that the new aliases “don’t work” — this is the single most common issue in this part; the fix is almost always just running `source ~/.zshrc`, not re-editing the file.
- A typo in the alias definition (mismatched quotes, a missing `=`) — can cause the *entire* `.zshrc`

to fail loading with an error on every new terminal open, which is a more serious problem than a single broken alias; if a student's terminal starts showing an error message on open after today's edits, walk through the added lines together character by character rather than letting it go unresolved.

Check for understanding: “If you typed `gs` in a folder that isn't a Git repository yet, what would happen?” (`git status` would report something like `fatal: not a git repository` — the alias still runs the underlying command exactly as typed; it has no awareness of whether that command will succeed in the current context, which is worth stating explicitly since it previews Week 4's actual Git work, where these aliases only become useful once inside an initialized repo.)

6.5 Part 4 — Practice and Deactivate (0:52–1:02, 10 min)

Teaching goal: Confirm pytest runs (even with no tests yet) inside the active venv, then deactivate cleanly and verify the return to system Python — closing the loop on Part 1’s activation.

Say to the class:

“Quick round-trip: confirm the venv still works for something real, then walk back out of the room and confirm you’re genuinely back to system Python — not just assume it.”

Live-code this:

```
cd ~/ism3232/module02_zsh
source .venv/bin/activate

# Quick test: pytest with no tests yet returns exit 0
pytest --collect-only    # should say 'no tests found'

# Deactivate
deactivate
which python3            # back to system Python
# (.venv) prefix is gone from the prompt
```

Line-by-line explanation:

- `pytest --collect-only` — runs pytest’s test-discovery step *without actually running anything* — since no test files exist yet in this project (that’s next module’s work), this should report finding zero tests, not an error. Say explicitly: this is a legitimate, useful check on its own — confirming pytest itself is correctly installed and runnable *before* you have real tests to run against it, so a broken install doesn’t get confused with a genuinely failing test later.
- `deactivate` — the exact reverse of `source .venv/bin/activate`; a built-in command that becomes available specifically *because* a venv is currently active — “walking back out of the room,” per the intro’s analogy.
- `which python3` (again) — should now report the **system** Python path again, not the `.venv` one — the concrete, checkable proof that deactivation genuinely reversed activation’s effect.
- The `(.venv)` prefix disappearing from the prompt — the same visual confirmation from Part 1, now confirming the opposite state.

Then, update the README with a new section:

```
cd ~/ism3232/module02_zsh
code README.md
```

Add a section titled `## Week 3: Virtual Environments` and `.zshrc` listing the commands and aliases learned today with one-line descriptions — the same documentation habit from Weeks 1–2, now accumulating into a single growing README rather than a fresh one each week.

Common student mistakes to watch for:

- Running `pytest --collect-only` **outside** the venv (having forgotten to activate, or having already deactivated) — this might still work if `pytest` happens to also be installed system-wide, which would mask the fact that the venv-specific install isn't actually being used; a good moment to double-check `which python3` immediately before running `pytest` if the room seems uncertain.
- Assuming `deactivate` also somehow deletes or breaks the venv itself — reassure explicitly: `deactivate` only affects the *current shell session's* configuration; the `.venv/` folder and everything installed in it remain completely intact, ready to be reactivated at any time with `source .venv/bin/activate` again.

Check for understanding: “You deactivated, closed your terminal, and came back the next day to keep working on this project. What's the very first command you should run?” (`source .venv/bin/activate`, from inside `module02_zsh/` — get a student to state this as the automatic first move for returning to *any* venv-based project, the same reflex as `pwd` in Week 1–2.)

6.6 Stretch — Recreate the Environment from `requirements.txt` (1:02–1:15, as time allows)

Frame as genuinely worth live demo time if the room reaches it — this is the single most convincing proof that `requirements.txt` actually works:

```
deactivate
rm -r .venv
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip list    # pytest and ruff should both appear
```

If you demo this live, narrate the `rm -r .venv` step specifically, since it's the first recursive delete of the semester: “`-r` means recursive — delete this folder *and everything inside it*. This is more dangerous than any single-file `rm` from Week 2, which is exactly why we're only doing it here, deliberately, on a folder we know is fully reproducible from `requirements.txt` — the whole point of this Stretch is proving that claim true before you'd ever trust it on something that matters.”

The payoff, worth stating explicitly once `pip list` shows both packages again: “Nothing about `pytest` or `ruff` was manually reinstalled by name — `pip install -r requirements.txt` read the file from Part 1 and recreated the *exact* environment, version numbers included, from scratch. This is why `requirements.txt` gets committed to Git while `.venv/` itself never does: the file is small, portable, and sufficient; the folder is neither.”

7 Wrap-Up (last ~13 minutes, folded into Stretch time if unused)

Review the submission checklist together, on screen:

- ☐ **Screenshot 1:** (.venv) prompt and which python3 output
- ☐ **Screenshot 2:** pip install and cat requirements.txt output
- ☐ **Screenshot 3:** ls -la showing both .venv/ and .gitignore
- ☐ **Screenshot 4:** ll, gs, and mkcd all working after source ~/.zshrc
- ☐ **File upload:** module02_zsh/requirements.txt
- ☐ **File upload:** module02_zsh/.gitignore
- ☐ **File upload:** module02_zsh/README.md with the Week 3 section added
- ☐ All uploaded to Canvas by end of class

Preview Week 4: “Today’s .gitignore and the five Git aliases you just added — gs, ga, gcmsg, gp, gl — are the exact tools next week’s lab uses to search your codebase and push your first real commit to GitHub. This was preparation; next week is the payoff.”

8 Appendix A — Full Command Reference

Part 1 (venv workflow):

```
cd ~/ism3232/module02_zsh
python3 -m venv .venv
source .venv/bin/activate
which python3
pip list
pip install pytest ruff
pip list
pip freeze > requirements.txt
cat requirements.txt
```

```
# in a SEPARATE, non-activated terminal tab:
which python3
pip list
```

Part 2 (.gitignore):

```
cd ~/ism3232/module02_zsh
echo '.venv/' > .gitignore
echo '__pycache__/' >> .gitignore
echo '*.pyc' >> .gitignore
cat .gitignore
ls -la
```

Part 3 (.zshrc):


```
# add to ~/.zshrc:
# ISM3232 required aliases
alias ll='ls -la'
alias c='clear'
alias py='python3'
alias gs='git status'
alias ga='git add .'
alias gcmsg='git commit -m'
alias gp='git push'
alias gl='git log --oneline'
alias tree2='tree -L 2'

# Required shell function
mkcd() { mkdir -p "$1" && cd "$1" }
```

```
source ~/.zshrc
ll
gs
mkcd testdir
pwd
```

Part 4 (practice + deactivate):

```
cd ~/ism3232/module02_zsh
source .venv/bin/activate
pytest --collect-only
deactivate
which python3
```

Stretch (rebuild from requirements.txt):

```
deactivate
rm -r .venv
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip list
```

9 Appendix B — Extra Practice (only if the class finishes early)

Four required parts plus the Stretch fill the full class period at a normal pace, including realistic environment-setup friction. If a section moves unusually fast:

Extra — a second alias, self-authored. Have students add one alias of their own choosing to .zshrc (e.g., `alias update='git pull'`, or something genuinely useful to their own workflow),

source ~/.zshrc, and test it — good extra rehearsal of the edit-source-test cycle on content they design themselves rather than copying a provided list.

Extra — a second mkcd-style function, read (not written). Show students this variant and have them predict what it does before testing it: `rmcd() { cd .. && rm -r "$1" }` — (goes up one level, then removes the named folder from the parent — have the room articulate why this ordering matters, and why combining `rm -r` with a shell function deserves the same caution as Part 3/Stretch's manual recursive delete, not less.)